

09 — CAP + HANA Sample Project: Architecture & Data Model (Deep Dive)

Module purpose: how the Python templates (sheets 01-08) map onto a real SAP **full-stack** project — CAP (Node.js) as backend, HANA as persistence, deployed as one MTA. Grounded in a real CAP + AI Core Orchestration project structure.

Project Layout

```
my-ai-app/
├─ db/schema.cds          <- data model (tables, as CDS entities)
├─ srv/
│   ├─ design-time.cds    <- service definitions (entities exposed + actions/functions)
│   └─ runtime.js         <- service implementation (business logic, AI Core calls – sheet 10)
├─ app/chat-ui/webapp/    <- SAPUI5 freestyle app (sheet 10)
├─ package.json           <- cds.requires: destinations, AI Core config
├─ xs-security.json       <- XSUAA roles/scopes
└─ mta.yaml               <- deployment descriptor (sheet 10)
```

Why this shape: CAP separates WHAT the data looks like (db/schema.cds) from WHAT the service exposes (srv/*.cds) from HOW it behaves (srv/*.js) — mirrors the Python templates' own separation (connect_env.py = data access, chatbot_core.py = behaviour, chatbot_config.py = declared settings).

Data Model — db/schema.cds

```
entity Conversation {
    key cID : UUID not null;           // primary key, server-generated on insert
    userID: String;                   // owner column – used for row-level security below
    creation_time: Timestamp;
    title: String;
    to_messages: Composition of many Message on to_messages.cID = $self; // 1-to-many, OWNED relationship
}

entity Message {
    key cID: Association to Conversation; // foreign key back to the parent conversation
    key mID: UUID not null;
    role: String;                       // "user" | "assistant"
    content: LargeString;               // unbounded text – CDS equivalent of HANA's NCLOB
    creation_time: Timestamp;
}

entity DocumentChunk : cuid, managed { // cuid = auto UUID key; managed = auto createdAt/createdBy/modifier
    text_chunk: LargeString;
    metadata_column: LargeString;
    embedding: Vector(1536);           // CDS-level equivalent of sheet 04's REAL_VECTOR(1536)
}
```

Working explanation: key cID : UUID not null is a **primary key** exactly like the "ID" NVARCHAR PRIMARY KEY in sheet 04's vector_table.py — CDS just declares it at a higher, DB-agnostic level and the CAP compiler generates the actual HANA DDL. Composition of many means **ownership** — deleting a Conversation cascades and deletes its Message rows automatically; Association (used for Message.cID pointing back to its parent) is a plain reference WITHOUT that ownership cascade. cuid and managed are CDS **aspects** — reusable mixins that inject a standard set of fields (ID, audit

timestamps) into any entity that includes them, avoiding repetition across every table needing the same boilerplate columns.

Service Definition — `srv/design-time.cds`

```
service ChatService @(requires: 'authenticated-user') { // <- XSUAA-protected: caller must have a valid JWT
  entity Conversation @(restrict: [
    {grant: ['READ','WRITE','DELETE'], where: 'userID = $user'} // <- ROW-LEVEL security, enforced by the framework
  ]) as projection on db.Conversation;

  action getChatRagResponse(user_query: String, conversationId: String) returns RagResponse; // custom only custom operation
  function deleteChatData() returns String; // read-only custom operation
}
```

Working explanation: entity `Conversation` as projection on `db.Conversation` is CAP's "auto-CRUD" mechanism — this ONE line exposes a full OData v4 endpoint (GET/POST/PATCH/DELETE with filtering, paging, sorting) with **zero controller code**, equivalent to writing an entire Flask/FastAPI CRUD router by hand in the Python world. The `@restrict` annotation with `where: 'userID = $user'` is enforced automatically on EVERY generated query — `$user` resolves to the authenticated caller's ID from their XSUAA token — this is framework-level row security, impossible to accidentally bypass by forgetting an `if` check in a new handler (unlike manually adding `WHERE userID = ?` in every Python route). action vs function: an action may have side effects (calling the LLM, writing data); a function is expected to be read-only — this distinction maps directly to HTTP semantics (POST vs GET) under the hood.

HANA Persistence — CQL (CDS Query Language)

```
// CAP's ORM-style query builder – compiles to parameterized SQL, conceptually identical to
↳ sheet 01/03's raw hdbcli SQL:
await INSERT.into(Conversation).entries({ cID, userID, title });
const messages = await SELECT.from(Message).where({ cID_cID: conversationId
↳ }).orderBy('creation_time');
await UPDATE(Conversation).set`title = ${newTitle}`.where`cID = ${conversationId}`;
await DELETE.from(Conversation).where({ cID: conversationId });
```

Working explanation: every CQL statement is automatically **parameterized** — the `${newTitle}` template-literal substitution is safe against SQL injection the same way `sheet 03's cursor.execute(sql, (params,))` placeholder binding is, just with JS template literal syntax instead of `?` placeholders. `SELECT.from(Message).where({cID_cID: conversationId})` compiles down to something functionally equivalent to `sheet 01's raw cursor.execute("SELECT * FROM MESSAGE WHERE CID = ?", (conversationId,))` — CQL is a higher abstraction over the exact same relational engine.

Security Model — `xs-security.json` (referenced by `mta.yaml`)

Defines the OAuth2 **scopes** and **role templates** the app needs (e.g. `Conversation.Read`, `Conversation.Write`) — XSUAA issues JWTs carrying these scopes after login, and `@(requires: 'authenticated-user')` plus `@restrict` checks against them at request time. This is the CAP-world counterpart to how you'd implement auth middleware/decorators in a Python Flask/FastAPI app, just declared once centrally instead of scattered across route handlers.

Key Points

- CAP separates **data model** (`db/schema.cds`), **service contract** (`srv/*.cds`), and **behaviour** (`srv/*.js`) — three concerns, three files, matching the Python templates'

connection/config/logic split.

- Composition = owned, cascading relationship; Association = plain reference, no ownership.
- entity ... as projection on db.X auto-generates full CRUD OData endpoints — no controller code.
- @restrict with where: 'userID = \$user' is framework-enforced row-level security — declared once, applied to every generated query automatically.
- Vector(1536) in CDS models the same HANA vector engine as sheet 04's raw REAL_VECTOR(1536), just at the modeling layer instead of hand-written DDL.
- action/function are how you expose **non-CRUD** operations (like “call the LLM”) through an otherwise auto-generated service — this is where runtime.js (sheet 10) plugs in custom logic.
- CQL (SELECT.from, INSERT.into) is parameterized by default — same SQL-injection protection as hdbcli's ? placeholders (sheet 01/03), different syntax.

Interview Q&A

Q: What's the practical difference between Association and Composition in CDS? A: Composition implies ownership and lifecycle dependency — deleting the parent cascades to delete children. Association is a plain foreign-key-style reference with no such cascade — deleting the referenced entity does not automatically delete the referencing rows.

Q: How does @restrict differ from checking userID manually inside a Python route handler? A: It's declared once at the model level and enforced by the CAP framework on every generated query for that entity — you cannot forget to add the check in a new endpoint, unlike scattered manual if row.userID != current_user checks that a future developer might omit.

Q: Why put getChatRagResponse as a CDS action instead of calling the LLM directly from the UI5 frontend? A: Keeps AI Core credentials/deployment IDs entirely server-side, lets you apply masking/filtering/auth consistently through the CAP layer, and keeps the frontend a thin OData client — the same separation-of-concerns principle as keeping AI Core calls out of any Streamlit/CLI presentation layer in the Python templates (sheet 06).

Q: How would you add server-side content filtering (like sheet 07) to this CAP service? A: Inside the getChatRagResponse action's implementation in runtime.js, call the Orchestration Service (via cap-llm-plugin or a direct HTTP call to the orchestration deployment) instead of a plain model call — same OrchestrationConfig concepts as sheets 07-08, just invoked from Node.js instead of Python.